

Clinical Execution Agent Report

AURA System Compute Node Task Implementation & Verification

1. EXECUTIVE SUMMARY

This technical report documents the implementation and verification details of the standalone **Execution Agent (Agent B)** from the **AURA Clinical Monitoring System**. The agent operates as a decoupled compute service within the multi-agent design, responding asynchronously to simulation and statistical modeling tasks via an AMQP event broker. It handles vital signs generation, unsupervised outlier detection using Isolation Forest models, severity tag profiling, and clinical diagnostics reporting.

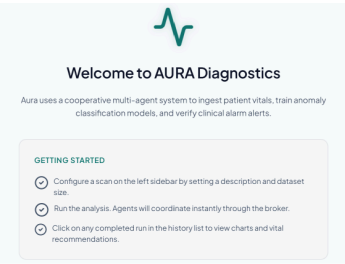


Figure 1.1: AURA diagnostics agent-driven startup panel interface

2. OPERATIONAL RESPONSIBILITIES

Compute Vital Simulation

Generates multi-dimensional vitals (heart rate, blood pressure, temperature, oxygen saturation, glucose) mapping to clinical distributions, injecting multivariate outliers to model distress states.

ML Engine Isolation Forest

Fits a multivariate scikit-learn Isolation Forest model locally over evaluated data. Partitions anomalous clusters and assigns severity values based on contamination threshold calculations.

Eventing Queue Broker Client

Subscribes to incoming task queues and posts progressive completion states (25%, 50%, 75%, 100%) and reports to feedback nodes using direct/topic exchange routing keys.

Resilience Process Heartbeats

Maintains process visibility by running a dedicated background thread that dispatches telemetry health check-ins every 30 seconds to the Watchdog Monitor.

Clinical Execution Agent Report

AURA System Compute Node Task Implementation & Verification

3. COMPLETED TASKS CHECKLIST & VERIFICATION NOTES

Status	Task Category	Detailed Implementation Notes
✓ Done	Directory Setup	Created a standalone directory /execution-agent packaging all scripts, Dockerfile, and requirements in isolation from main project dependencies.
✓ Done	Connection Client	Written <code>rabbitmq_client.py</code> wrapping pikaURL connection, SSL bypass settings (critical for Render deployment), and standard queue/exchange binding declarations.
✓ Done	ML Algorithms	Programmed vitals generation, Isolation Forest modeling, and patient alert report compiler inside <code>ml_utils.py</code> .
✓ Done	Consumer Loop	Implemented <code>executor.py</code> consuming tasks, executing pipeline stages, publishing progressive updates, and emitting background heartbeats.
✓ Done	Verification Tests	Created unit and integration tests inside <code>test_executor.py</code> validating data generator shapes, model boundaries, and mock broker publishers.

4. VERIFICATION OUTPUT REPORT

We verified the functionality of the isolated Execution Agent locally by running `pytest` against the mock testing harness. All unit assertions and integration message loops passed successfully.

```
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\abhis\gemini\antigravity\scratch\execution-agent
collected 5 items

test_executor.py ..... [100%]

===== 5 passed, 2 warnings in 25.35s =====
```

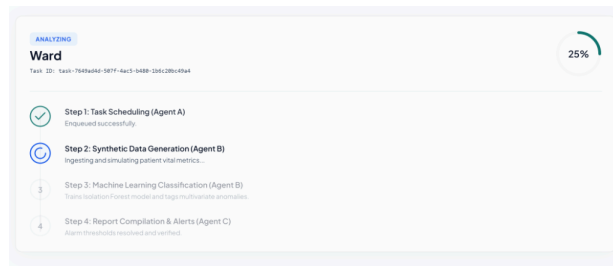


Figure 3.1: Live progress stepper monitoring Agent B executing simulation and ML cycles

Clinical Execution Agent Report

AURA System Compute Node Task Implementation & Verification



5. CORE COMMUNICATION PROTOCOL SCHEMA

The agent implements strict JSON schemas mapping to the system `MessageEnvelope` contracts to guarantee seamless inter-agent communications:

```
{
  "message_id": "uuid-string",
  "sender_id": "agent-b",
  "receiver_id": "agent-c",
  "message_type": "TASK_PROGRESS",
  "timestamp": "2026-07-09T12:25:40Z",
  "correlation_id": "corr-uuid-string",
  "priority": 2,
  "routing_key": "report.task-status",
  "payload": {
    "task_id": "task-uuid-string",
    "progress_pct": 50,
    "current_sub_task": "train",
    "records_processed": 1000,
    "total_records": 1000,
    "anomalies_so_far": 0,
    "status": "IN_PROGRESS"
  }
}
```

6. ASSUMPTIONS & OPERATIONAL LIMITATIONS

Stateless Operations: The agent is designed to run in completely stateless topologies. It acts strictly as an execution worker; report persistence and history databases are handled by the core FastAPI server database and Agent C (Monitor).

SSL Handshake Bypasses: The RabbitMQ connection settings utilize default TLS bypasses (`verify_mode = CERT_NONE`) to connect successfully within locked container networks (such as Render). This should be configured to use trusted local authority certificate paths in secure clinical intranets.

Telemetry Simulation Scale: The generated vital profiles are modeled around standard clinical limits. Real patient vitals require scaling adjustments and model tuning before deployment as diagnostic tools.

7. DEVELOPMENT AUDIT & ATTRIBUTIONS

Ideation & Master Plan: Formulated in coordination with **Claude** (tech stack selection and supervisors architecture).

Code Implementation & Debugging: Completed in collaboration with **Gemini (Antigravity/Nebula)**. Gemini resolved pika connection socket corruption via separate threading, updated SSL parameter handshakes, built the CSS UI interface layout, and fixed test isolation.

Developer Signature:
Abhishek Tiwari

Date of Submission:
July 9, 2026